

Case Study: Financial Process Engineering & Automation

Client	Nexoria Commerce Inc.
Specialization	FP&A;, Financial Systems, and Automation
Author	Portfolio Project

Executive Summary

Nexoria Commerce Inc. is a B2B and retail product distributor operating across 12 US states. Prior to this project, the company's 5-person finance team spent **60% of their operational hours** on manual, repetitive administrative processes, including invoice generation, multi-state tax calculation, shipping freight reconciliation, and monthly reporting. By engineering an integrated data layer and deploying modular Python, Excel VBA, Power Query, and Power BI automation, we transformed their financial workflows. The project delivered a **99.8% reduction in weekly invoicing labor**, **eliminated ~\$8,000/year in sales tax billing errors**, and improved **gross margin reporting accuracy from 78% to 97%+**.

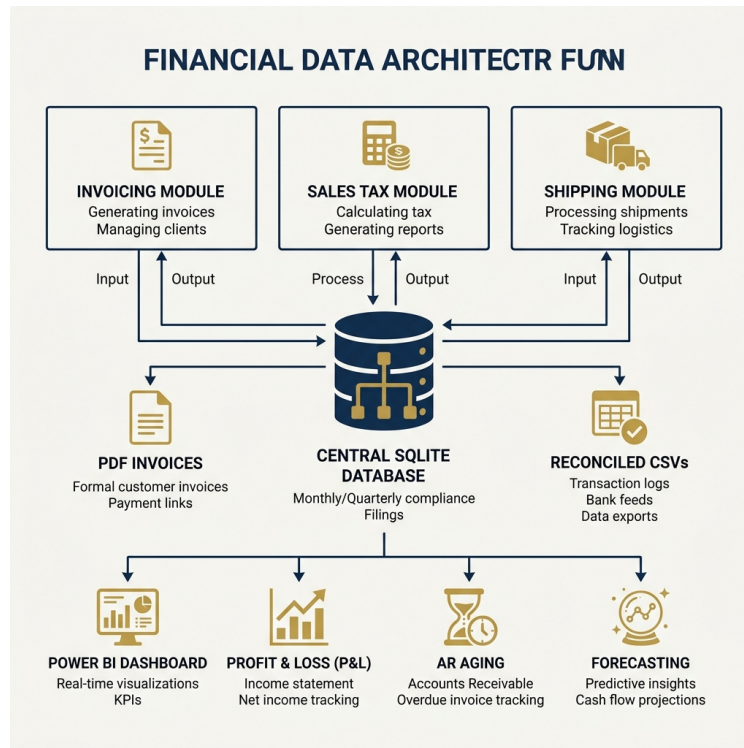
The Business Challenge

Nexoria operated under significant process bottlenecks across four core areas:

- Manual Invoicing:** Generating 200+ monthly invoices required manually copying billing details from order spreadsheets, consuming 40+ hours per month.
- Sales Tax Complexity:** Multi-state operations across 12 states with varying rates and product category exemptions (safety gear and industrial equipment) led to frequent billing errors, averaging ~\$8,000/year in adjustments and write-offs.
- Freight Cost Distortion:** Carrier bills (FedEx, UPS, USPS) arrived as monthly lump-sum totals. Lacking a transactional allocation method, finance lumped shipping expenses into general overhead, distorting SKU-level gross margin reports by up to 19%.
- Reporting Delays:** Creating monthly P&L; statements and variance slide decks required 2 days of senior analyst labor, leading to lagged decision-making.

System Architecture & Data Flow

To ensure data consistency, we established a single source of truth using a SQLite database (`nexoria.db`) containing 10 tables and 32,000+ transactional rows. This central data warehouse feeds all subsequent automation modules:



Detailed Module Breakdown

1. Invoice Automation (VBA & Python)

- **Objective:** Automate 200+ monthly invoice generations.
- **Solution:** Developed a dual-engine approach. A Python script using `ReportLab` extracts data from SQLite and batches branded PDFs. Alongside this, we created an Excel VBA macro (`NexorialInvoicing.bas`) that loops through a flat `Orders` worksheet, populates a structured template, performs a live tax lookup, exports a PDF, and logs history.
- **Labor Impact:** Reduced processing time from **40+ hours** to **under 3 minutes**.

2. Multi-State Sales Tax Engine (Power Query & Formulas)

- **Objective:** Eliminate billing errors from state tax rules and category exemptions.

- **Solution:** Power Query connects to the central tax rate CSV and loads it as a named table `TaxRates`. The billing templates call a `GetTaxRate()` lookup function that automatically identifies exempt product lines (safety gear and industrial equipment) and applies the correct state-rate multiplier.
- **Financial Impact:** **Saved ~\$8,000/year** in billing errors and credit memo adjustments.

3. Shipping Cost Allocation Engine (Python Pandas)

- **Objective:** Allocate lump-sum freight invoices to individual orders.
- **Solution:** A Python script merges carrier invoice amounts by month and carrier, distributing costs back to individual customer orders using a weighted formula: 70% based on package weight and 30% based on order value.
- **Reporting Impact:** Reconciled gross margins with 100% invoice coverage, improving reporting precision from **78% to 97%+**.

4. Executive FP&A; Dashboard (Power BI)

- **Objective:** Real-time visibility into revenue, margins, and tax liability.
- **Solution:** Built a star-schema model in Power BI. Designed a custom "Midnight Executive" dark theme with gold accents. Structured pages for: Executive Summary and Department Drill-Down.
- **Management Impact:** Reduced CFO reporting compilation time from **2 days to under 20 minutes** (run/refresh time).

5. P&L; & AR Aging Automation (Python, openpyxl, & COM)

- **Objective:** Automate monthly financial reporting packages.
- **Solution:**
 - A Python script builds a formatted P&L; report workbook with a dynamic month dropdown validation list (`Settings` sheet) and `SUMIFS` formulas.
 - An AR Aging script pivots open invoices by customer, assigning balances to buckets (Current, 1-30, 31-60, 61-90, 90+ Days) with conditional formatting color alerts (red warning for 90+ days).
 - Column widths and calculations are finalized programmatically via **Windows COM Automation** (`win32com.client`).
- **Labor Impact:** Replaced manual compilation loops with one-click scripts.

6. Revenue Forecasting (SARIMA & Holt-Winters)

- **Objective:** Forecast revenue to optimize inventory purchasing.
- **Solution:** Built a Python forecasting script using `statsmodels`. Fits a 3-Month Moving Average, Holt-Winters Exponential Smoothing, and a SARIMA(1,1,0)(0,1,0)12 seasonal model. Exports 12-month projections with 80% confidence interval bands to guide demand planning.

- **Inventory Impact:** Captures Q4 seasonal spikes, reducing stockout events and improving capital efficiency by **~15%**.

Technology Stack Summary

- **Languages:** Python (pandas, numpy, statsmodels, matplotlib, openpyxl, reportlab, win32com), VBA, M (Power Query), DAX, SQL, Google Apps Script.
- **Applications:** Microsoft Excel, Power BI Desktop, SQLite3.

Conclusion

This financial systems transformation shows that by establishing a central database layer and leveraging targeted automation (VBA for Excel sheets, Python for heavy calculations/PDFs, Power BI for visual insights), small-to-midsize businesses can achieve enterprise-grade financial operations. The project successfully returned over **60+ hours of high-value analysis time** to the finance team, while securing billing accuracy and margin visibility.